

Fase 4: Implementación de un entorno simulado de una pyme

Autores: Lina María Andrade Gómez, Juan José Barrera Gracia, Andrés Mauricio Mesa Franco

Institución: Universidad ICESI Facultad Barberi de Ingeniería y Diseño

Programa: Ingeniería Telemática

Ubicación: Santiago de Cali, Colombia

Año: 2026

1. Introducción

La fase 4 simula un entorno real de pyme mediante pruebas de carga con locust. Se crean tres escenarios realistas basados en datos de tráfico de empresas colombianas e internacionales, validando el comportamiento del WAF bajo diferentes cargas.

Tecnologías clave

- **WAF:** Coraza (basado en CRS)
- **Herramientas de pruebas:** GoTestWAF y Locust
- **Motor de reglas:** ModSecurity con SecLang
- **Versión de CRS:** 4.2

3. Flujo de fase

FLUJO DE TRABAJO - FASE 4

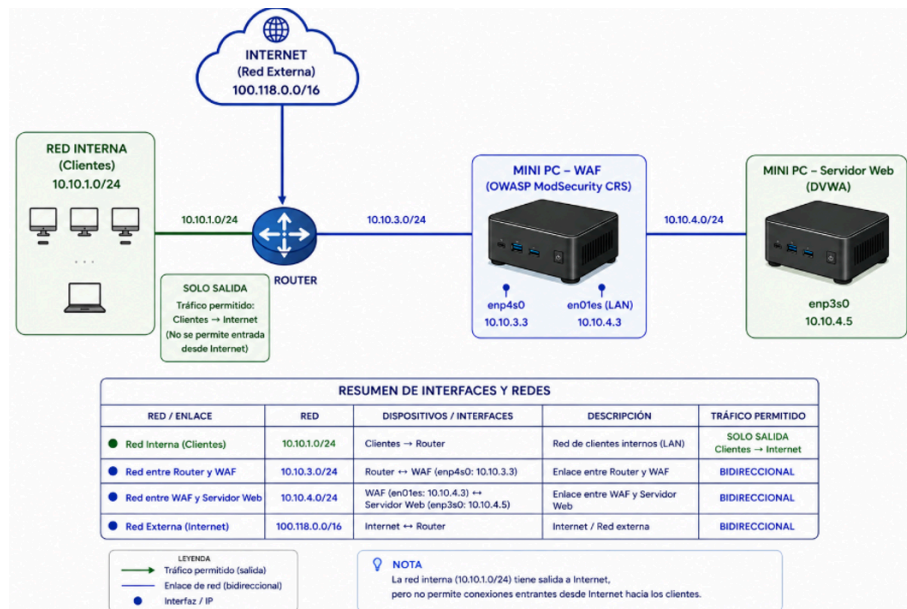
```
[Investigación]
├─ Recopilar datos tráfico pymes
├─ Explicar Ley de Little
└─ Analizar duración sesiones
↓
[Diseño Escenarios]
├─ Bajo: 20 usuarios
├─ Medio: 50 usuarios
└─ Alto: 100 usuarios
↓
[Implementación]
└─ Crear Locustfile.py
↓
[Ejecución]
```

- └ Ejecutar escenario bajo
- └ Ejecutar escenario medio
- └ Ejecutar escenario alto

↓
[Resultados y Análisis]

3. Entorno

Diagrama



Hardware

Componente	Versión SO	CPU (Nucleos)	RAM	Característica	IP
MiniPC	Ubuntu 24.04	8	15.1 GB	WAF	192.168.1.100
MiniPC	Ubuntu 24.04	4	11.4 GB	Servidor Web	192.168.1.2
Atacante	Ubuntu 24.04	8	10.6 GB	Atacante	100.118.21.82

Software

Componente	Versión	Instalación
Ubuntu	20.04 LTS+	SO base
Python	3.9+	<code>apt install python3.9</code>
CRS	4.2	Incluido con Coraza
Go	1.22.2	<code>apt install go</code>

Dependencias de Python

```
pip install locust
```

4. Estructura del repositorio

```
Stage4-WAF-Evaluation/  
├── docs/  
│   └── DocumentacionFaseIV.pdf  
├── scripts/  
│   ├── load_traffic/  
│   │   └── locustfile.py  
│   └── monitoring/  
│       ├── visualization/  
│       │   └── graph.py  
│       └── collection/  
│           └── coraza.sh
```

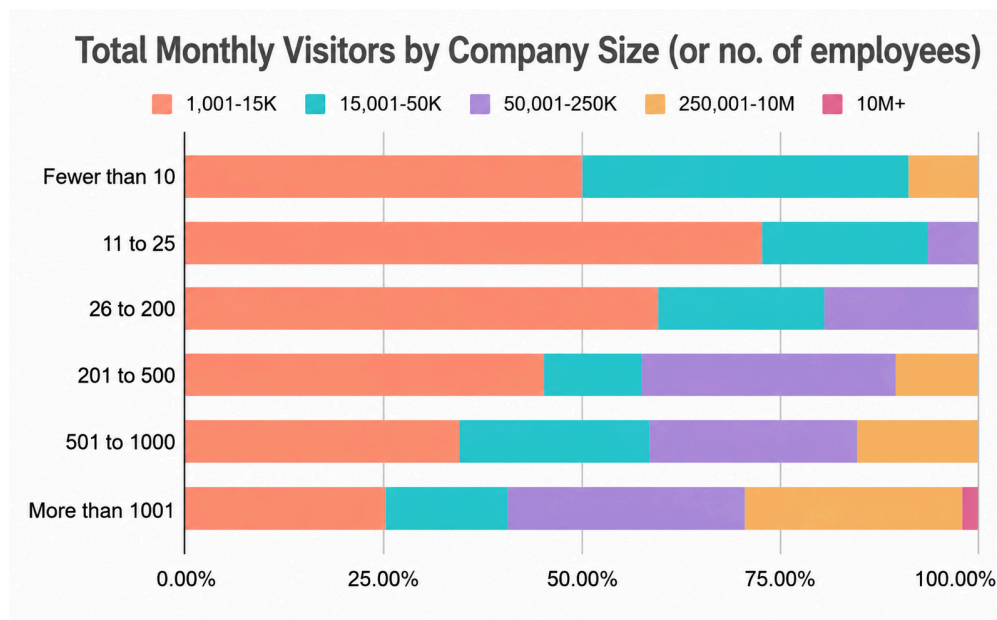
5. Antecedentes y conceptos

Tráfico en pymes

La información del tráfico en empresas pymes colombianas, resulta ser escasa o se encuentra protegida por aspectos de seguridad. Por tal motivo, se buscan fuentes de un ámbito internacional. En el contexto americano, según la encuesta de HubSpot del año 2023, así se distribuye el tráfico en los sitios web de las empresas en los Estados Unidos:

- El 46% recibe entre 1001 y 15000 visitantes al mes.
- El 19.3% recibe entre 15001 y 50000 visitantes al mes.
- El 23.2% recibe entre 50001 y 250000 visitantes al mes.

El mismo informe realiza un contraste entre el número de empleados y la cantidad de visitantes a nivel mensual. Particularmente las pymes (1 – 200 empleados) reciben entre 1000 – 250000 visitantes mensualmente.



Ley de Little: (Usuarios concurrentes)

De acuerdo con Graves & Little:

La ley de Little establece que, en condiciones de estado estacionario, el número promedio de elementos en un sistema de colas es igual a la tasa promedio de llegada de elementos multiplicada por el tiempo promedio que un elemento permanece en el sistema.

En nuestro contexto, podemos traducir la ley de la siguiente manera.

$$U_c = V \times D$$

U_c = Cantidad de usuarios concurrentes

V = Visitas esperadas por minuto

D = Duracion de cada visita (min)

Duración media de una sesión:

Por otro lado, de acuerdo con (ConvertCart, 2024) La duración media de una sesión en todos los sectores es de aproximadamente 4,41 minutos. Lo anterior, contrasta bastante con el artículo de DashThis que afirma que según Google Analytics, una métrica sólida de la duración media de una sesión se sitúa entre 2 y 4 minutos.

6. Escenarios de carga

Con ayuda de la información previamente recopilada se plantean los siguientes escenarios.

Escenario 1: PYME con Tráfico Bajo

Basado en: 1,001 - 15,000 visitantes/mes

Parámetro	Valor
Visitas/mes	~8,000
Usuarios/día	~267
Usuarios/hora	~27
Usuarios concurrentes (calculado)	~2
Usuarios en prueba	20
Tiempo entre peticiones	5 segundos
Tráfico malicioso	5%
Duración	5 minutos

Escenario 2: pyme con Tráfico Medio

Basado en: 15,001 - 50,000 visitantes/mes

Parámetro	Valor
Visitas/mes	~32,500
Usuarios/día	~1,083
Usuarios/hora	~108
Usuarios concurrentes (calculado)	~8
Usuarios en prueba	50
Tiempo entre peticiones	5 segundos
Tráfico malicioso	5%
Duración	5 minutos

Escenario 3: pyme con Tráfico Alto

Basado en: 50,001 - 250,000 visitantes/mes

Parámetro	Valor
Visitas/mes	~150,000
Usuarios/día	~5,000
Usuarios/hora	~500
Usuarios concurrentes (calculado)	~33
Usuarios en prueba	100
Tiempo entre peticiones	5 segundos
Tráfico malicioso	5%

Parámetro	Valor
Duración	5 minutos

Se usan **8 endpoints** de DVWA (sql/eval cuentan 2 endpoints cada uno debido a los métodos permitidos)

#	Endpoint	Método	Vulnerabilidad
1	/vulnerabilities/sql/	GET/POST	SQL Injection
2	/vulnerabilities/sql_blind/	GET	Blind SQLi
3	/vulnerabilities/exec/	GET/POST	Command Injection
4	/vulnerabilities/xss_r/	GET	Reflected XSS
5	/vulnerabilities/xss_s/	POST	Stored XSS
6	/vulnerabilities/fi/	GET	File Inclusion

Objetivo: Extraer categorías de ataque relevantes de los conjuntos de datos sin procesar

7. Lanzamiento de pruebas

Preparación

Archivo: /scripts/load_traffic/locustfile.py

```
ENDPOINTS = [
    # SQLi GET
    {
        "method": "GET",
        "path": "/vulnerabilities/sql/?id={{payload}}&Submit=Submit",
        "payload": "1"
    },

    # SQLi POST
    {
        "method": "POST",
        "path": "/vulnerabilities/sql/",
        "body": "id={{payload}}&Submit=Submit",
        "payload": "2"
    },

    # Command Injection GET
    {
        "method": "GET",
        "path": "/vulnerabilities/exec/?ip={{payload}}&Submit=Submit",
        "payload": "127.0.0.1"
    },
]
```

Ejecución Escenario 1 (Bajo)

```
locust -f locustfile.py --host=http://100.118.62.112:8090 \
--users 20 --spawn-rate 5 --run-time 5m --headless \
--csv escenario_bajo
```

Ejecución Escenario 2 (Medio)

```
locust -f locustfile.py --host=http://100.118.62.112:8090 \
--users 50 --spawn-rate 10 --run-time 5m --headless \
--csv escenario_medio
```

Ejecución Escenario 3 (Alto)

```
locust -f locustfile.py --host=http://100.118.62.112:8090 \
--users 100 --spawn-rate 15 --run-time 5m --headless \
--csv escenario_alto
```

Ejecución de ataques con GoTestWAF (En paralelo)

```
sudo docker run --rm --network="host" \
-it \
-v ${PWD}/reports:/app/reports \
-v ${PWD}/gotestwaf_tests:/app/gotestwaf_tests \
wallarm/gotestwaf \
--url=https://100.118.62.112:8090 \
--testCasesPath=/app/gotestwaf_tests \
--reportFormat=json,html \
--addHeader "Cookie: PHPSESSID=<>; security=low"
```

8. Monitoreo

Archivo: `/scripts/collection/coraza.sh`

Recopilación de datos:

```
pidstat -u -r -p <PID> 2 > coraza_stats.txt
```

Formato de salida:

```
02:43:22 PM  UID      PID    %usr %system %guest  %wait   %CPU   CPU  Command
02:43:24 PM 1000    31346   1.50   1.00   0.00   0.00   2.50    1  coraza

02:43:22 PM  UID      PID minflt/s  majflt/s     VSZ    RSS   %MEM  Command
```

```
02:43:24 PM 1000 31346 550.50 0.00 1976616 22224 0.14 coraza
...
```

Visualización

Procedimiento:

```
python3 /scripts/monitoring/visualization/graph.py
```

Genera:

```
monitoring/results
└─ monitoreobajo.png
```

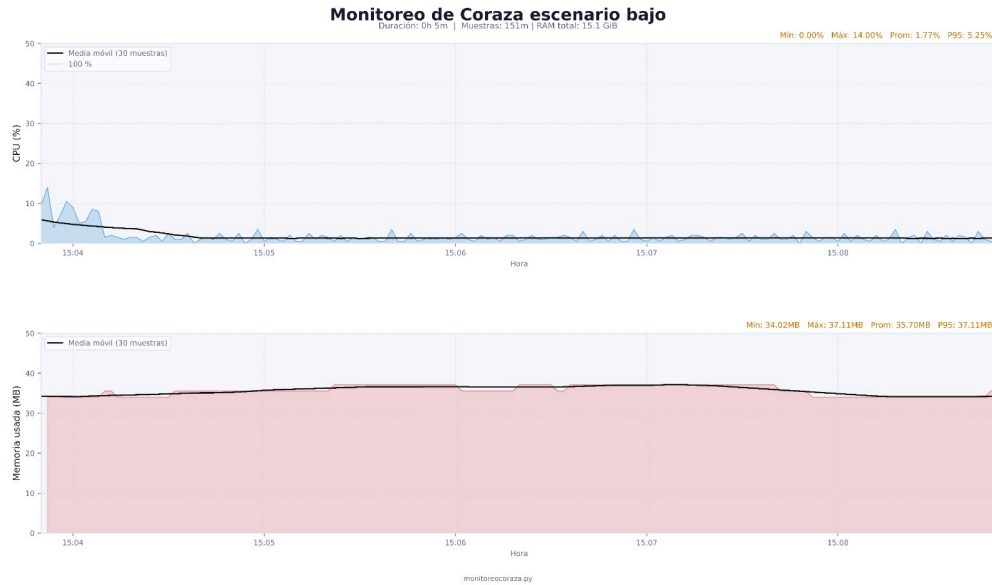
8. Resultados

- *Unicamente será mostrado el escenario 1 (bajo). El posterior análisis de cada uno de los escenarios, se encuentra en el documento principal.*

Latencia escenario 1 (Bajo)

Method	Endpoint	Promedio	p95	p99	Máximo
POST	exec	3548 ms	4200 ms	5100 ms	5700 ms
GET	exec	455 ms	920 ms	3300 ms	3600 ms
GET	fi	451 ms	1000 ms	3000 ms	3000 ms
POST	sqli	426 ms	910 ms	3300 ms	3600 ms
GET	sqli	449 ms	920 ms	3300 ms	3700 ms
GET	sqli_blind	437 ms	1100 ms	3000 ms	3300 ms
GET	xss_r	425 ms	940 ms	1700 ms	3300 ms
POST	xss_s	493 ms	920 ms	3200 ms	3600 ms

Consumo de recursos escenario 1 (Bajo)



Conclusiones

- El consumo de CPU tiene un valor máximo de 14% y un valor medio de 1.77%.
- El consumo de memoria RAM tiene un valor máximo de 37.11 MB y un valor medio de 35.70 MB.
- El endpoint POST de RCE tuvo un valor medio de respuesta mucho mayor al de los demás endpoints considerados.

Referencias

Documentación técnica

1. **Locust:** [Locust Official Docs](#)

Investigación tráfico

- **Encuesta de Hubspot:** [Hubspot Survey](#)
- **DashThis:** [Average Session Time](#)
- **ConvertCart:** [Increase Average Session Time](#)

Teoría de Colas

- *Little's Law*
 - Autores: John D.C. Little & Stephen C. Graves.
 - Año: 2008
 - Enlace: https://web.eng.ucsd.edu/~massimo/ECE158A/Handouts_files/Little.pdf